08/11/2025

SORTING:

SORTING IS A FUNDAMENTAL ALGORITHM DESIGN PROBLEM. MANY EFFICIENT ALGORITHM USE IT, BECAUSE MAKES THE DATA PROCESSING EASIER.

FOR EXAMPLE, "DOES AN ARRAY CONTAIN DUPLICATES?", BECOMES EASY WITH SORTING BECAUSE 2 EQUAL ELEMENTS WOULD BE NEXT TO EACH OTHER AFTER SORTING.

THE EFFICIENT GENERAL SORTING ALGORITHMS WORK IN $O(n \log n)$.

SORTING THEORY:

THE BASIC PROBLEM: "GIVEN AN ARRAY THAT CONTAINS n ELEMENTS, YOUR TASK IS TO SORT THE ELEMENTS IN INCREASING ORDER.

1	3	8	2	9	2	5

$\Rightarrow$

1	2	2	3	5	8	9

$O(n^2)$ ALGORITHM:

A FAMOUSE $O(n^2)$ ALGORITHM IS BUBBLE SORT WHERE THE ELEMENTS "BUBBLE" IN THE ARRAY ACCORDING TO THEIR VALUE.

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n-1; j++) {
        if(arr[j] > arr[j+1]) {
            // swap arr[j] and arr[j+1]
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}
```

$O(n \log n)$ ALGORITHM:

IS IT POSSIBLE TO SORT AN ARRAY IN $O(n \log n)$ TIME USING ALGORITHMS THAT ARE NOT LIMITED TO SWAPPING CONSECUTIVE ELEMENTS. ONE SUCH ALGORITHM IS MERGE SORT, WHICH IS BASED ON RECURSION. MERGE SORT SORTS A SUBARRAY ARRAY$[a \ldots b]$ AS FOLLOWS:

1) IF $a == b$, DO NOT DO ANYTHING BECAUSE THE SUBARRAY IS ALREADY SORTED.
2) CALCULATE THE POSITION OF THE MIDDLE ELEMENT: $K = [(a+b)/2]$.
3) RECURSIVELY SORT THE SUBARRAY ARRAY$[a \ldots K]$.
4) RECURSIVELY SORT THE SUBARRAY ARRAY$[K+1 \ldots b]$.
5) MERGE THE SORTED SUBARRAY ARRAY$[a \ldots K]$ AND ARRAY$[K+1 \ldots b]$ INTO ARRAY$[a \ldots b]$.

Sorting in C++:

It is almost never a good idea to use a home-made sorting algorithm, C++ standard library contains the function sort: → saves time.

```
vector<int> v = {4,3,5,3,5,8,3};
sort(v.begin(), v.end());
```

After sorting the vector will be [3,3,3,4,5,5,8]. The default order increasing, but a reverse order is possible:

```
vector<int> v = {4,3,5,3,5,8,3};
sort(v.rbegin(), v.rend());
```

An ordinary array can be sorted as follows:

```
int n = 7;  // array size
int arr[] = {3, 5, 2, 9, 6, 1, 4};
sort(arr, arr + n);
```

The following code sorts the string s:

```
string s = "monkey";
sort(s.begin(), s.end());
```

→ "ekmnoy"

Comparison operator:

The function sort requires that a comparison operator is defined for the data type. For example, numbers are sorted according to their values and strings are sorted in alphabetical order. Pairs are sorted primarily according to their first element (first), however, if the first elements of two pairs are equal, they are sorted according to their second elements (second).

```
vector<pair<int, int>> v;
v.push_back({1, 5});
v.push_back({2, 3});
v.push_back({1, 2});
sort(v.begin(), v.end());
```

→ (1,2), (1,5), (2,3).

In similar way the tuples (tuple):

```
vector<tuple<int, int, int>> v;
v.push_back({2, 1, 4});
v.push_back({1, 5, 3});
v.push_back({2, 1, 3});
sort(v.begin(), v.end());
```

→ (1,5,3), (2,1,3), (2,1,4)

User-defined struct:

User-defined struct do not have a comparison operator automatically. The operator should be defined inside the struct as a function "operator<", whose parameter is a element of the same type. For example, the following struct P contains the x and y coordinates of a point, the comparison operator is defined that the points are sorted primarily by the x coordinate and secondarily by the y coordinate.

```
struct P {
    int x, y;
    bool operator<(const P &p) {
        if(x != p.x) return x < p.x;
        else return y < p.y;
    }
}
```

Comparison functions:

It is also possible to give an external comparison function to the sort function as a callback function. For example, the following comparison function comp sorts strings primarily by length and secondarily by alphabetical order.

```
bool comp(string a, string b) {
    if(a.size() != size(b)) return a.size() < a.size();
    return a < b;
}
```

Now a vector of string can be sorted as follows:

```
sort(words.begin(), words.end(), comp);
```

Binary Search:

A general method for searching for an element is to use a for loop that iterates through the array:

```
for (int i = 0; i < n; i++){
    if(array[i] == x) {
        // x found at index i
        break;
    }
}
```

$\rightarrow O(n)$

However, if the array is sorted, we can use a much faster algorithm, the binary search algorithm that can search in a sorted array in $O(\log n)$ time

```
int a = 0, b = n-1;
while (a <= b){
    int k = (a + b) / 2;
    if(arr[k] == x){
        // x found at index k
    }
    if(array[k] > x) b = k - 1;
    else a = k + 1;
}
```

0 $k = \frac{6+0}{2} = 3$ 6

1	2	3	4	5	6	7
a						b

$x = 2$

$4 == x$? NO

$4 > x$? YES $\Rightarrow b = k - 1$

$k = \frac{3}{2} = 1$

1	2	3
a		b

$2 == x$? YES $\Rightarrow$ AT ARRAY [K]

METHOD 2:

AN ALTERNATIVE METHOD TO IMPLEMENT BINARY SEARCH IS BASED ON AN EFFICIENT WAY TO ITERATE THROUGH THE ELEMENT OF THE ARRAY

```
int k = 0;
for (int b = n/2; b >= 1; b /= 2) {
    while (k + b < n && arr[k + b] <= x) {
        k += b; // Move k up by b
    }
}
if (arr[k] == x) {
    return k; // Element found at index k
}
```

DURING THE SEARCH, THE VARIABLE b CONTAINS THE CURRENT JUMP LENGHT.

C++ FUNCTIONS:

THE FOLLOWING FUNCTIONS ARE BASED ON BINARY SEARCH:

- LOWER_BOUND: RETURNS A POINTER TO THE FIRST ELEMENT WHOSE VALUE IS AT LEAST X.
- UPPER_BOUND: RETURNS A POINTER TO THE FIRST ELEMENT WHOSE VALUE IS LARGER THAN X.
- EQUAL_RANGE: RETURNS BOTH ABOVE POINTERS.

THE FUNCTIONS ASSUME THAT THE ARRAY IS SORTED.

IF THERE IS NO SUCH ELEMENT IT RETURNS THE POINTER THAT POINTS OUTSIDE THE ARRAY:

```
auto k = lower_bound(array, array+n, x)-array;
if(k < n && array[k] == x){
    // x found at index k
}
```

THEN, THE FOLLOWING CODE COUNTS THE NUMBER OF ELEMENT WHOSE VALUE IS X:

```
auto a = lower_bound(array, array+n, x);
auto b = upper_bound(array, array+n, x);
cout << b-a << "\n";
```

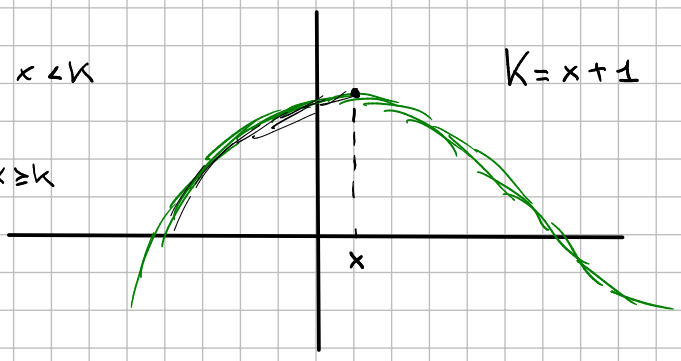
USING EQUAL_RANGE:

```
auto r = equal_range(array, array+n, x);
cout << r.second - r.first << "\n";
```

FINDING THE MAXIMUM VALUE:

$f(x) < f(x+1)$ WHEN $x < K$

$f(x) > f(x+1)$ WHEN $x \geq K$




```
int x = -1;
for(int b = n/2; b >= 1; b /= 2){
    while(f(x+b) < f(x+b+1)) x += b;
}
int k = x+1;
```